Distributed Task Orchestrator

Backend Assignment (Microservices + Saga Pattern)

Overview

Implement a fault-tolerant task orchestrator for workflow automation (e.g., a Pune fintech's KYC pipeline: Verify ID → Credit check → Approve loan). Use Node microservices with Kafka for orchestration; handle 10k workflows/hour.

Business Context & Value: Enterprise systems rarely execute within a single monolithic transaction. Complex workflows (like loan approvals, travel bookings, or order fulfillments) require coordination across multiple isolated microservices. An orchestrator acts as the "director" ensuring that all steps succeed, or failing that, ensures all completed steps are logically rolled back (compensated) to maintain data consistency without sacrificing the benefits of decentralized microservices.

Real-world complexities:

- **Handling Saga failures (partial rollbacks):** If step 3 fails, how do you undo the side-effects of steps 1 and 2 robustly?
- **Service downtime (implementing circuit breakers):** Preventing cascading failures when a downstream API is unreachable.
- **State replay from an event log:** Recovering the exact state of incomplete workflows after an orchestrator crash.

Exact Problem Statement & Expected Workflow

Your primary objective is to build the FlowMaster orchestration system. Here is exactly what you need to do:

1. **Microservice Topology Setup:** Create at least three minimal Node.js Express applications. One acts as the Orchestrator, the second as the 'Verification' service, and the third as the 'Credit' service. They should not communicate directly via HTTP, but rather implicitly via publishing events to Kafka topics.
2. **Saga Orchestrator Logic:** The Orchestrator service must read a YAML definition of a workflow. It should trigger the first step by publishing an event (e.g. `start_verification`) and wait for the verification service to publish a success/failure result. If successful, it triggers the next step (`start_credit_check`).
3. **Compensation (Rollback) Engineering:** You must deliberately implement a failure scenario. If the credit check service fails or rejects the user, the Orchestrator must detect this and immediately dispatch a 'compensation' event back to the Verification service to undo or flag the previous successful step, maintaining system-wide consistency.

Embed Square Solutions Pvt. Ltd.

Gokhale Business Bay, A-1111, Paschimnagari, Kothrud, Pune - 411038, Maharashtra, India.
www.embeds2.com

4. **State Persistence:** The Orchestrator must log the exact state of every workflow in Postgres. If the Orchestrator crashes mid-workflow, upon restarting, it should read the Postgres event log and resume the workflow exactly from the step where it left off.

Complications:

- **Sagas:** Choreography via Kafka topics; outbox pattern for reliability.
- **Scale:** Shard workflows by ID; use async retries (e.g., BullMQ).
- **Resilience:** Use Hystrix-like breakers; ensure replay functionality from a Postgres event store.
- **Monitoring:** Implement Jaeger traces and Prometheus metrics.

Tech Stack

- **Microservices:** Node.js/Express (minimum 3 microservices: Orchestrator, ID, Credit).
- **Infrastructure:** Kafka (Message Broker), Postgres (Events/Storage), Redis (State caching).

Tasks / Functional Requirements

5. **YAML Parser:** Allow developers to define saga steps independently.
6. **Executor Application:** Publish/consume events over Kafka domains; actively compensate if sub-tasks fail.
7. **API Setup:** `/workflows` endpoints (submit entirely new workflows and query the current status).

Deliverables

- **Live codebase/repository:** `FlowMaster_[YourName]`.
- **README.md:**
  - Complete Saga diagram illustrating failure paths.
  - Failure recovery demo instructions.

Evaluation (~150 points)

- **Focus:** Idempotency guarantees, and end-to-end latency measuring <500ms.
- **Bonus:** Serverless (AWS Lambda) adaptation in place of permanent Node instances.

Good luck — we look forward to reviewing your work!

Embed Square Solutions Pvt. Ltd.

Gokhale Business Bay, A-1111, Paschimnagari, Kothrud, Pune - 411038, Maharashtra, India.
www.embeds2.com